EXPERIMENT 8

AIM: To include libraries and work with files in a Flutter application.

OBJECTIVE: To add an external package, locate application storage, create and manage a text file, display its content asynchronously, and handle file-system errors.

COURSE OUTCOME: CO 3 and CO 4 - Integrate data persistence using file handling and evaluate reliable application behavior through error handling and verification.

THEORY

Flutter packages are declared in pubspec.yaml. The path_provider package supplies platform-aware directories without hard-coded device paths. After adding the dependency, flutter pub get resolves and installs the package, and its API can be imported in Dart source files.

getApplicationDocumentsDirectory() returns a location intended for files generated by the application. The dart:io File class creates, writes and reads files asynchronously. Platform.pathSeparator keeps the constructed path portable across supported operating systems.

FutureBuilder connects asynchronous file operations to the widget tree. It can show a progress indicator while reading, display file text after completion, and present a useful message when an exception occurs. try-catch blocks handle missing files, permission failures and other file-system errors.

SOFTWARE REQUIREMENTS

- Flutter SDK with Dart 3.10 or later
- Android Studio or Visual Studio Code
- Android emulator, physical device, Windows, macOS or Linux desktop
- path_provider package version 2.1.6

CODE

pubspec yaml

```
name: experiment_8_file_handling
description: Experiment 8 - External libraries and file handling in Flutter.
publish_to: "none"
version: 1.0.0+1

environment:
  sdk: ">=3.10.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  path_provider: ^2.1.6

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0

flutter:
  uses-material-design: true
```

main dart

```
import 'package:flutter/material.dart';

import 'home_screen.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const FileHandlingApp());
}

class FileHandlingApp extends StatelessWidget {
  const FileHandlingApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'File Handling in Flutter',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF0F6B5B),
        ),
        useMaterial3: true,
      ),
      home: const HomeScreen(),
    );
  }
}
```

file service dart

```
import 'dart:io';

import 'package:path_provider/path_provider.dart';

class FileService {
  static const String fileName = 'experiment_8_notes.txt';

  Future<Directory> get documentsDirectory async {
    final directory = await getApplicationDocumentsDirectory();
    print('Application documents directory: ${directory.path}');
    return directory;
  }

  Future<File> get localFile async {
    final directory = await documentsDirectory;
    return File('${directory.path}${Platform.pathSeparator}$fileName');
  }

  Future<String> get filePath async => (await localFile).path;

  Future<File> writeText(String text) async {
    try {
      final file = await localFile;
      return await file.writeAsString(text, flush: true);
    } on FileSystemException catch (error) {
      throw Exception('Unable to write the file: ${error.message}');
    } catch (error) {
      throw Exception('Unexpected write error: $error');
    }
  }

  Future<String> readText() async {
    try {
      final file = await localFile;
      if (!await file.exists()) {
        await writeText(
          'Welcome to Experiment 8!\n'
          'This text was created and stored in the app documents folder.',
```

```
        );
      }
      return await file.readAsString();
    } on FileSystemException catch (error) {
      throw Exception('Unable to read the file: ${error.message}');
    } catch (error) {
      throw Exception('Unexpected read error: $error');
    }
  }
}
```

home screen dart

```
import 'package:flutter/material.dart';

import 'file_service.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  final FileService _fileService = FileService();
  final TextEditingController _textController = TextEditingController(
    text: 'Flutter makes local file handling simple and reliable.',
  );

  late Future<String> _fileContent;
  late Future<String> _filePath;
  bool _isSaving = false;

  @override
  void initState() {
    super.initState();
    _reloadFile();
  }

  @override
  void dispose() {
    _textController.dispose();
    super.dispose();
  }

  void _reloadFile() {
    setState(() {
      _fileContent = _fileService.readText();
      _filePath = _fileService.filePath;
    });
  }

  Future<void> _saveFile() async {
    final text = _textController.text.trim();
    if (text.isEmpty) {
      _showMessage('Enter some text before saving.', isError: true);
      return;
    }

    setState(() => _isSaving = true);
    try {
      await _fileService.writeText(text);
      if (!mounted) return;
      _reloadFile();
      _showMessage('Text file saved successfully.');
    } catch (error) {
      if (!mounted) return;
      _showMessage(error.toString(), isError: true);
    } finally {
      if (mounted) setState(() => _isSaving = false);
```

```
  }
}

void _showMessage(String message, {bool isError = false}) {
  ScaffoldMessenger.of(context)
    ..hideCurrentSnackBar()
    ..showSnackBar(
      SnackBar(
        content: Text(message),
        backgroundColor: isError ? Colors.red.shade700 : null,
      ),
    );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Experiment 8'),
      centerTitle: true,
    ),
    body: SafeArea(
      child: ListView(
        padding: const EdgeInsets.all(20),
        children: [
          Text(
            'Local File Notebook',
            style: Theme.of(context).textTheme.headlineMedium?.copyWith(
                  fontWeight: FontWeight.bold,
                ),
          ),
          const SizedBox(height: 6),
          const Text(
            'Write text to the application documents directory and read it asynchronously.',
          ),
          const SizedBox(height: 20),
          TextField(
            key: const Key('noteInput'),
            controller: _textController,
            minLines: 3,
            maxLines: 5,
            decoration: const InputDecoration(
              labelText: 'Text to store',
              hintText: 'Enter file content',
              prefixIcon: Icon(Icons.edit_note_rounded),
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 12),
          FilledButton.icon(
            key: const Key('saveButton'),
            onPressed: _isSaving ? null : _saveFile,
            icon: _isSaving
                ? const SizedBox.square(
                    dimension: 18,
                    child: CircularProgressIndicator(strokeWidth: 2),
                  )
                : const Icon(Icons.save_rounded),
            label: Text(_isSaving ? 'Saving...' : 'Write to text file'),
          ),
          const SizedBox(height: 24),
          Row(
            children: [
              const Expanded(
                child: Text(
                  'Saved File Content',
                  style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
                ),
              ),
              IconButton(
                key: const Key('refreshButton'),
                tooltip: 'Read file again',
                onPressed: _reloadFile,
                icon: const Icon(Icons.refresh_rounded),
              ),
            ],
```

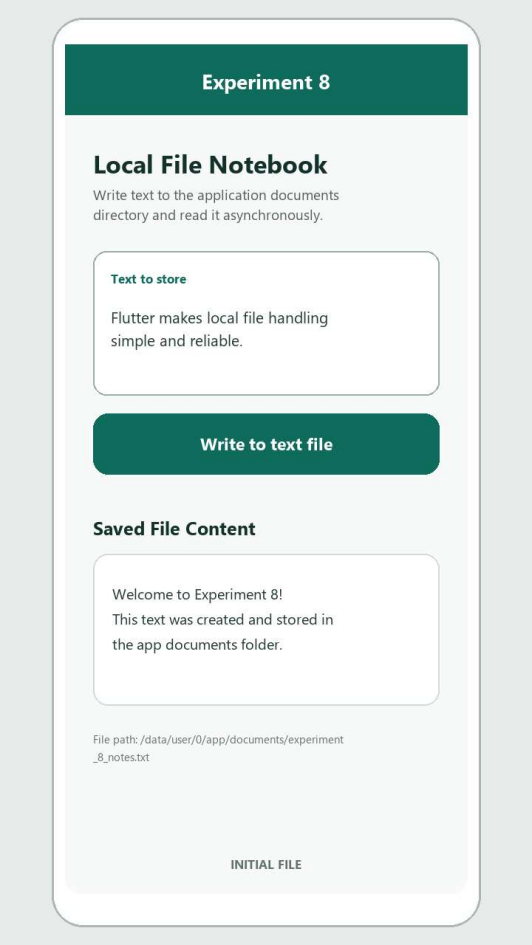
```
            ),
            FutureBuilder<String>(
              future: _fileContent,
              builder: (context, snapshot) {
                if (snapshot.connectionState == ConnectionState.waiting) {
                  return const Card(
                    child: Padding(
                      padding: EdgeInsets.all(28),
                      child: Center(child: CircularProgressIndicator()),
                    ),
                  );
                }

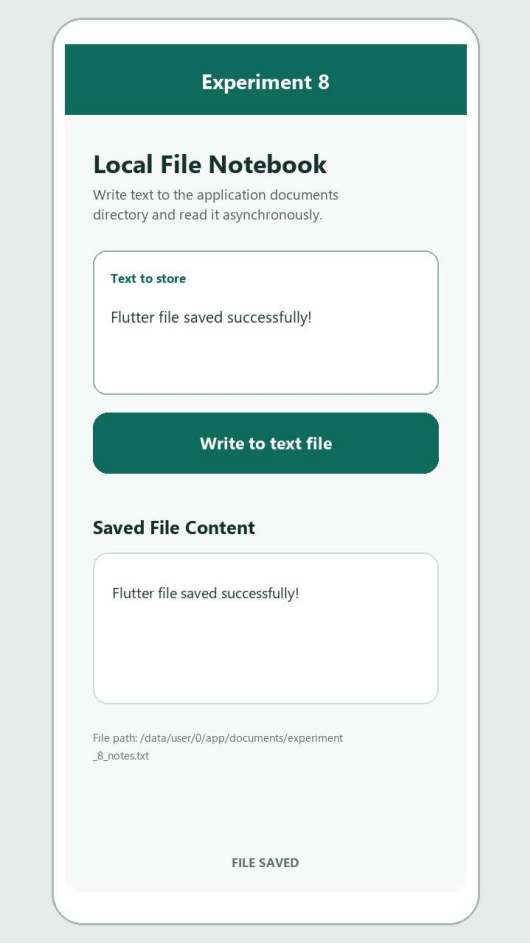
                if (snapshot.hasError) {
                  return Card(
                    color: Theme.of(context).colorScheme.errorContainer,
                    child: Padding(
                      padding: const EdgeInsets.all(18),
                      child: Text(
                        'File error: ${snapshot.error}',
                        key: const Key('errorText'),
                      ),
                    ),
                  );
                }

                return Card(
                  child: Padding(
                    padding: const EdgeInsets.all(18),
                    child: SelectableText(
                      snapshot.data ?? 'The file is empty.',
                      key: const Key('fileContent'),
                      style: const TextStyle(fontSize: 16, height: 1.5),
                    ),
                  ),
                );
              },
            ),
            const SizedBox(height: 12),
            FutureBuilder<String>(
              future: _filePath,
              builder: (context, snapshot) {
                return Text(
                  snapshot.hasData
                      ? 'File path: ${snapshot.data}'
                      : 'Locating application documents directory...',
                  style: Theme.of(context).textTheme.bodySmall,
                );
              },
            ),
          ],
        ),
      ),
    );
  }
}
```

OUTPUT

The initial launch creates sample content when the text file is absent. After the user writes new text, FutureBuilder reloads the file and displays the updated content and storage path.





RESULT

A Flutter application was implemented with the path_provider library and dart:io file operations. The source locates the application documents directory, creates and updates a text file, reads its content asynchronously, and reports errors in the user interface.

CONCLUSION

This experiment demonstrated how an external Flutter package can provide a safe storage path and how dart:io supports persistent text files. FutureBuilder kept the interface responsive while the file was loaded, and structured exception handling protected the application from missing-file and file-system failures. The implementation addresses CO 3 and CO 4 at the Create level.